Sidecar Architectures for Portable Identity Systems

The Portable Identity Framework defines how identity may exist independently of a specific embodiment, while Identity-Constrained Character Robotics demonstrates one implementation of that principle within robotics. This paper addresses a separate problem: deployment.

Many existing systems cannot be rewritten to adopt a new cognitive architecture. Game engines contain established AI frameworks, robots contain existing control stacks, and conversational systems often depend upon large language models or domain-specific pipelines. Replacing these systems entirely is frequently impractical.

This paper introduces the Sidecar Architecture, a deployment model that allows a Portable Identity to operate alongside existing software and hardware systems without requiring ownership of execution. The sidecar observes system state, evaluates conditions through a Portable Identity, and dispatches behavior recommendations through interfaces already supported by the host environment.

The architecture allows identity-driven behavior to be integrated into robots, games, virtual worlds, and conversational systems while preserving existing execution infrastructure.

Introduction

The first challenge of character systems is identity creation.

The second challenge is identity portability.

The third challenge is deployment.

Existing platforms already contain mature execution systems responsible for navigation, movement, physics, rendering, simulation, dialogue generation, networking, and hardware control. Replacing these systems is often more expensive than extending them.

The Sidecar Architecture addresses this challenge by separating character identity from execution ownership.

Rather than replacing a host system, the sidecar operates alongside it. The host remains responsible for execution while the sidecar remains responsible for identity-driven decision support.

This separation allows Portable Identity systems to be introduced incrementally rather than through complete platform replacement.

Sidecar Architecture

The Sidecar Architecture consists of three functional layers.

Telemetry Layer

The telemetry layer observes the host system.

Examples include:

- actor state
- health values
- positional information
- dialogue context
- inventory state
- resource levels
- sensor readings

The telemetry layer is observational. It does not modify host state.

Its purpose is to provide the sidecar with the information necessary to evaluate the current situation.

Identity Evaluation Layer

The evaluation layer interprets telemetry through the Portable Identity.

The Portable Identity supplies:

- values
- preferences

- tendencies
- prohibitions
- relationship structures
- behavioral priorities

The evaluation layer determines which actions best align with the character's authored identity under current conditions.

The sidecar therefore answers a different question than a planner or pathfinding system.

Instead of asking:

> What can be done?

it asks:

> What would this character prefer to do?

Intent Dispatch Layer

The dispatch layer converts evaluated decisions into actions understood by the host.

The sidecar never performs movement, rendering, dialogue generation, or hardware actuation directly.

Instead, it communicates through interfaces already supported by the host environment.

This allows deployment without modifying the underlying execution architecture.

Integration Characteristics

A defining feature of the Sidecar Architecture is that it does not assume ownership of execution.

The host remains responsible for:

- animation
- navigation
- pathfinding
- physics simulation
- dialogue generation

- rendering
- hardware control

The sidecar remains responsible for:

- identity interpretation
- behavioral preference evaluation
- character consistency
- identity-driven decision support

Because responsibilities remain separated, the sidecar may be removed without disabling the host system.

The host returns to its native behavior while preserving operational integrity.

Conversational Systems

In conversational systems, the sidecar operates between user interaction and final response delivery.

The language model functions as a response generator rather than the character itself.

Generated candidates are evaluated against the Portable Identity before delivery.

This process allows:

- identity consistency
- character-specific speech patterns
- behavioral alignment
- preference preservation

without modifying the underlying language model.

Portable Identity therefore controls character expression while the language model continues providing linguistic generation.

Game Systems

In games, the sidecar operates alongside existing AI frameworks.

Behavior Trees, State Machines, Utility Systems, planners, and navigation systems continue performing their existing responsibilities.

The sidecar evaluates available actions through the Portable Identity and influences which actions receive priority.

This allows character behavior to remain portable across different engines and genres.

In strategy games, identity may influence:

- expansion priorities
- risk tolerance
- force preservation
- resource allocation
- diplomatic preferences
- tactical aggression

In role-playing games, identity may influence:

- combat style
- interaction preferences
- negotiation tendencies
- ally selection
- response to threats

The sidecar affects behavior without requiring replacement of the host game's AI architecture.

Robotic Systems

In robotics, the sidecar evaluates information supplied by sensors, body models, and control systems.

The architecture separates behavioral intent from physical execution.

Identity influences:

- action preferences

- interaction style
- response selection
- behavioral priorities

while existing safety systems remain responsible for:

- motion control
- balance
- actuator protection
- collision prevention
- hardware constraints

This separation allows different identities to share the same physical platform while preserving consistent character behavior.

Addon Architecture

The Sidecar Architecture supports extensibility through isolated addons.

Addons expand capability without modifying the Portable Identity.

Supported extension categories include:

- sensors
- equipment
- motion primitives
- rule configurations
- environmental interfaces

A sensor addon may provide new information.

A motion addon may provide new actions.

An equipment addon may expand available tools.

A rule addon may alter the constraints of a specific environment.

Identity remains separate from all of these modifications.

As a result, a character may gain new capabilities without becoming a different character.

Human Interaction and Player Experience

The purpose of the Sidecar Architecture is not simply deployment.

Its purpose is to preserve recognizable character behavior across repeated interactions.

Cooperative Characters

Many companion systems require extensive player supervision or contribute only limited tactical value.

A character operating through a Portable Identity behaves according to consistent priorities and preferences.

Over time, players learn:

- what the companion values
- which risks the companion accepts
- how the companion responds to pressure
- when the companion becomes cautious
- when the companion becomes aggressive

Cooperation therefore becomes based on understanding character behavior rather than repeatedly issuing commands.

Adversarial Characters

Identity-driven opponents derive differences from authored priorities rather than statistical advantages alone.

Different opponents encourage different forms of play.

A defensive opponent rewards patience and positional control.

An aggressive opponent rewards spacing and interruption.

A cautious opponent rewards pressure.

A reckless opponent rewards counterattacks.

Players adapt not only to mechanics but to the behavior of individual opponents.

Persistent World Characters

The architecture is equally applicable to non-combat populations.

Merchants, guards, travelers, citizens, trainers, and companions can exhibit consistent decision-making patterns across repeated encounters.

These behavioral differences allow players to recognize characters through actions rather than exposition alone.

Over time, recurring interactions can create familiarity while preserving individual variation.